ATTENDEASE · SOFTWARE ENGINEERING MINI PROJECT

Test Case Document

Document ID	AE-TCD-01
Version	2.0
Date	10 November 2026
Build under test	`main` @ `92bcd23` (merge of PR #113, 1 Nov 2026)
Prepared by	Arundhati K (PES1UG24CS083)
Team	Arundhati K, Anshdeep Singh Sachdeva, Atharva Ashish Vyas, Bhoomika Dayanand Jituri

Step-by-step test cases TC-01 to TC-13, traceability and execution results.

1. Document control

All 13 test cases pass on the final build (`main` @ `92bcd23`, 10 November 2026). This document expands the 13 representative cases of the Test Plan (§13) into step-by-step cases, aligned with the implemented system and the recorded deviations.

Item	Detail
Document ID	AE-TCD-01
Version	2.0 (replaces the Test Plan v1.0 §13 table)
Project	AttendEase, Software Engineering Mini Project
Build under test	`main` @ `92bcd23` (merge of PR #113, 1 Nov 2026)
Prepared by	Arundhati K (PES1UG24CS083)
QA lead	Bhoomika Dayanand Jituri (PES1UG25CS807)
Related documents	SRS v1.2, SAD v1.0, Test Plan, Unit Test Report AE-UTR-01, Code Coverage Report AE-CCR-01, Branch Coverage Report AE-BCR-01, System Validation Report AE-SVR-01, `deviations.md`

Revision history

Version	Date	Change
2.0	10 Nov 2026	Cases written as executable steps with endpoints and test data; expected results aligned with the implementation and deviations D-01 to D-11; automated tests and execution results added.
1.0	03 Oct 2026	13 representative cases in the Test Plan §13 table.

What changed from v1.0

No expected result was weakened. The changes make each case precise enough to execute.

TC	Change in v2.0	Reason
TC-01	"Confirmation shown" means 201 Created, returning the account id and role and never the password. Hashing is checked in the database (PBKDF2-SHA256). SEC-001 added.	The API has no UI page yet; SEC-001 is the hashing requirement.
TC-02	Threshold fixed at 5 failed attempts within 15 minutes. While locked, login returns 429 Too Many Requests, even with the correct password.	Values from `users/security.py`.
TC-03	"Faculty dashboard" is checked through `GET /api/academic/my-sections`.	D-04: the faculty web UI is not built; the endpoint returns the same data.
TC-04	The session is started for a scheduled timetable period, on today's or a past date.	D-06 (session linked to a period), NFR-004 / D-07 (no future dates).
TC-05	Duplicate is sent through the batch endpoint `POST …/records`. The error body uses DRF's default format.	FR-037 scope; D-02 (error format). The per-student toggle may change a mark while the session is open (D-08).
TC-06	Percentages count closed sessions only, and are checked against an independent recount of the student's history.	FR-029 rule; open sessions are not yet final.
TC-07	The email is checked with Django's console email backend.	No SMTP server in the test environment; the same code path runs.
TC-08	Approval is `PATCH /api/corrections/{id}` with `{"action":"approve"}`. `acted_by` and `acted_at` are set by the server, never by the client.	SEC-006.
TC-09	"Action logged" means a WARNING log entry naming the user, role, view and method. A second admin-only endpoint was added.	SEC-003 covers every write endpoint.

TC	Change in v2.0	Reason
TC-10	The threshold is the course's own (FR-035, default 75%). Percentages are checked against an independent SQL recount. FR-029 added.	Per-course thresholds were implemented.
TC-11	Measured twice: in the automated suite on every CI build, and as a 50-user load test against the running server (Gunicorn).	D-09.
TC-12	Statement and branch coverage reported separately, on the 7 service-layer files enforced by CI.	NFR-006 wording; CI's figure combines the two.
TC-13	Requests for another student's record return 404 (nothing disclosed) or 403. Both meet the expected result.	Not revealing that a record exists is stricter than 403.

2. Test environment and data

Every case runs against a live server over HTTP, with real JWT logins. The automated tests listed for each case run on every pull request in GitHub Actions.

Item	Detail
Server	Gunicorn, 1 sync worker (same command as `Dockerfile` / `docker-compose.yml`)
Software	Python 3.11 (CI) / 3.13, Django 4.2.30, DRF 3.17.2, SimpleJWT 5.5.1
Database	SQLite with all migrations (PostgreSQL in the Docker deployment)
Email	Django console backend; outgoing mail is read from the server log
Tools	Newman (Postman CLI) with `AttendEase_SystemValidation.postman_collection.json`; Python load generator (TC-11); coverage.py 7.17.0 (TC-12)
Data setup	`python manage.py migrate`, `createsuperuser`, `python demo_seed.py`, plus the load-test seed for TC-11

Test data

Account / item	Role	Used in
`admin@attendease.com`	Admin	TC-03
`faculty@attendease.com` (Dr. Meera Rao)	Faculty, assigned to CS341-A and CS342-A	TC-04 to TC-10
`ananya@`, `karthik@`, `priya@`, `rahul@`, `rohan@`, `sneha@attendease.com`	Students in CS341-A (all six) and CS342-A (first five)	TC-04 to TC-10, TC-13
`varun@`, `isha@attendease.com`	Students, not enrolled	TC-02
`neha@attendease.com`, `kavya@attendease.com`	Created during TC-01 (student) and TC-03 (faculty)	TC-01, TC-03
CS341 / CS342	4 weeks of closed sessions (20); Rohan and Sneha below 75% in CS341	TC-06, TC-10
Pending corrections	Ananya (to approve), Sneha	TC-08
CS399-L	80 enrolled load-test students, an open session, `perf.faculty@attendease.com`	TC-11

This exceeds the Test Plan §6 minimum (1 admin, 2 faculty, 10 students, 2 courses). Every demo password is `Demo@1234`; the admin password is set locally. The cases run in order on one database: TC-04 creates the session used by TC-05, and TC-06 counts it after TC-05 closes it.

3. Test cases

Each case lists its steps as API calls in the order the validation collection runs them. Results are from the run of 10 November 2026; evidence is in the System Validation Report (AE-SVR-01).

TC-01 · Register a new student account

Field	Detail
Requirements	FR-001, SEC-001
Type / priority	Functional, security / High
Preconditions	Server running; `neha@attendease.com` not registered
Expected result	201 Created with the account id and role; no password in the response; the account can log in; the stored password is a salted one-way hash.
Automated test	`users.tests.RegistrationTests.test_tc01_register_student_with_hashed_password`
Actual result	201 with id and role `student`, no password field; login 200; stored as `pbkdf2_sha256$600000$….`
Status	PASS

1. `POST /api/auth/register` with name Neha Gupta, email `neha@attendease.com`, password, role `student`, USN `PES1UG24CS020`.
2. Check the response: 201, a numeric `id`, `role` = `student`, no `password` field.
3. `POST /api/auth/login` with the same email and password; expect 200 and a token.
4. In the database, check that the stored password starts with `pbkdf2_sha256$` and does not contain the plain password.

TC-02 · Lockout after repeated failed logins

Field	Detail
Requirements	SEC-002, FR-003
Type / priority	Security / High
Preconditions	`varun@attendease.com` exists with no recent failed logins
Expected result	After 5 failed attempts in 15 minutes, login is refused with 429 until the cooldown ends, even with the correct password. Other accounts are unaffected.
Automated test	`users.tests.LoginTests.test_tc02_failed_logins_lock_account_for_cooldown`
Actual result	Attempts 1–5: 401. Correct password on attempts 6 and 7: 429, no token. `isha@` logs in normally (200).
Status	PASS

1. Send `POST /api/auth/login` for `varun@attendease.com` with a wrong password 5 times; expect 401 each time.
2. Send the correct password; expect 429 and no `access` token.
3. Send the correct password again; expect 429.
4. Log in as `isha@attendease.com`; expect 200.

TC-03 · Admin assigns a faculty member to a section

Field	Detail
Requirements	FR-013
Type / priority	Functional / High
Preconditions	Admin account; faculty `kavya@attendease.com` registered in step 1
Expected result	Assignment saved; the faculty member sees the section in their section list (D-04).
Automated test	`academic.tests.FacultyAssignmentTests.test_tc03_admin_assigns_faculty_who_then_sees_section`
Actual result	Course and section 201, assignment 200; `my-sections` lists exactly CS360-A.
Status	PASS

1. Register faculty Prof. Kavya Rao; log in as admin.
2. `POST /api/academic/courses` with code CS360, title Cloud Computing; expect 201.
3. `POST /api/academic/courses/{id}/sections` with code A; expect 201.
4. `POST /api/academic/assignments` with the section id and faculty id; expect 200.
5. Log in as Kavya; `GET /api/academic/my-sections`; expect only CS360 section A.

TC-04 · Start a session and bulk-mark all present

Field	Detail
Requirements	FR-014, FR-016, FR-017
Type / priority	Functional / High
Preconditions	Faculty assigned to CS341-A, which has a period scheduled today and 6 enrolled students
Expected result	The session and its attendance records are created; after the single bulk action the roster shows every student as present.
Automated test	`attendance.test_sessions.MarkAllPresentTests.test_tc04_start_session_then_bulk_mark_all_present`
Actual result	Session 201 (open); mark-all-present 200; the roster lists 6 students, all present.
Status	PASS

1. Log in as `faculty@attendease.com`; `GET /api/academic/my-sections` and pick today's CS341 period.
2. `POST /api/attendance/sessions/start` with `timetable_slot_id`; expect 201 and `closed` = false.
3. `POST /api/attendance/sessions/{id}/mark-all-present`; expect 200.
4. `GET /api/attendance/sessions/{id}/roster`; expect 6 students, each `present`.

TC-05 · Duplicate attendance record rejected

Field	Detail
Requirements	FR-037
Type / priority	Functional / High
Preconditions	TC-04 done: the session has a record for every student
Expected result	The second attempt is rejected with 409 Conflict and the existing record is not changed.
Automated test	`attendance.tests.AttendanceSubmissionTests.test_tc05_repeated_submission_returns_409_without_overwrite`
Actual result	409; the roster still has one entry for Karthik, still present.
Status	PASS

1. `POST /api/attendance/sessions/{id}/records` with Karthik as `absent`; expect 409.
2. `GET …/roster`; expect exactly one entry for Karthik, still `present`.
3. `POST /api/attendance/sessions/{id}/close`; expect 200 (prepares TC-06).

TC-06 · Student attendance dashboard

Field	Detail
Requirements	FR-020, FR-021
Type / priority	Functional / High
Preconditions	Rohan is enrolled in CS341-A and CS342-A with closed sessions
Expected result	The dashboard shows the correct attendance percentage for each enrolled course, and the per-course breakdown.
Automated tests	`attendance.test_dashboard.StudentDashboardTests.test_tc06_course_percentage_counts_only_closed_sessions`, … `test_tc06_session_history_is_closed_and_student_scoped`
Actual result	Both courses listed. Each percentage equals present ÷ closed sessions from his history, and the below-threshold flags are correct.
Status	PASS

1. Log in as `rohan@attendease.com`.
2. `GET /api/attendance/dashboard/history?from=2026-01-01&to=<today>`; count present and total sessions per course.
3. `GET /api/attendance/dashboard`; expect CS341 and CS342, each with that percentage (±0.01) and `below_threshold` = percentage < threshold.

TC-07 · Student raises a correction request

Field	Detail
Requirements	FR-024, FR-025
Type / priority	Functional / High
Preconditions	Karthik has an absent record in CS341-A, with no open request on it
Expected result	The request is created with Pending status, and the assigned faculty member is notified by email.
Automated tests	`corrections.tests.CorrectionSubmissionTests.test_tc07_valid_request_is_pending_and_leaves_attendance_unchanged`, `corrections.test_notifications.CorrectionNotificationTests.test_tc07_success_sends_one_email_only_to_assigned_faculty`
Actual result	201 with status PENDING; the request is in the faculty queue; one email went to `faculty@attendease.com` only.
Status	PASS

1. Log in as `karthik@attendease.com`.
2. `POST /api/corrections` with `record_id` and a reason; expect 201 and `status = PENDING`.
3. As faculty, `GET /api/corrections`; expect the request, PENDING.
4. In the server mail log, expect one message to `faculty@attendease.com` naming Karthik.

TC-08 · Faculty approves a correction request

Field	Detail
Requirements	FR-027, SEC-006
Type / priority	Functional, security / High
Preconditions	Ananya has a pending request on a record marked absent; the faculty member is assigned to that section
Expected result	The attendance record is updated, the request is marked Approved, and `acted_by` and `acted_at` are populated on the CorrectionRequest.
Automated test	`corrections.test_approval.CorrectionApprovalTests.test_tc08_approval_updates_attendance_and_records_server_audit_fields`
Actual result	200, APPROVED; `acted_by` is the faculty member's id and `acted_at` a server timestamp (also in the database); Ananya's record is now present.
Status	PASS

1. As faculty, `GET /api/corrections`; take Ananya's request id and session id.
2. `GET /api/auth/me`; note the faculty member's id.
3. `PATCH /api/corrections/{id}` with `{"action":"approve"}`; expect 200, `status = APPROVED`, `acted_by` = that id and `acted_at` set.
4. `GET /api/attendance/sessions/{session}/roster`; expect Ananya `present`.
5. In the database, check `status`, `acted_by_id` and `acted_at` on the request.

TC-09 · Student denied an admin-only endpoint

Field	Detail
Requirements	SEC-003, FR-004
Type / priority	Security / High
Preconditions	Rohan is logged in as a student
Expected result	403 Forbidden is returned and the attempt is logged.
Automated tests	`academic.tests.CourseTests.test_tc09_student_calling_real_admin_endpoint_is_denied`, `users.tests.RoleAccessTests.test_tc09_student_cannot_call_admin_only_endpoint`
Actual result	Both calls 403, nothing created. Logged as `Role access denied: user_id=7 role=student view=CourseListView method=POST` (and the same for AssignmentView).
Status	PASS

1. As Rohan, `POST /api/academic/courses` with code CS998; expect 403.
2. As Rohan, `POST /api/academic/assignments`; expect 403.
3. In the server log, expect one `Role access denied` warning per call, with user, role, view and method.

TC-10 · Defaulter report for a section

Field	Detail
Requirements	FR-029, FR-031
Type / priority	Functional / High
Preconditions	CS341-A with closed sessions; threshold 75% (course default)
Expected result	The report lists exactly the students below the threshold, with correct percentages.
Automated test	`analytics.tests.DefaulterReportApiTests.test_report_lists_only_below_threshold_with_required_columns`
Actual result	Lists Rohan Das (53.85%) and Sneha Kulkarni (69.23%), matching a recount of all 6 students from the database.
Status	PASS

1. As faculty, `GET /api/reports/defaulters?section_id=<CS341-A>`; expect 200.
2. Expect exactly Rohan Das and Sneha Kulkarni, each with a percentage below their threshold.
3. Recount each enrolled student's percentage (present ÷ closed sessions) from the database; expect the report's list and values to match.

TC-11 · Performance under 50 concurrent users

Field	Detail
Requirements	NFR-001, NFR-002
Type / priority	Performance / High
Preconditions	CS399-L has 80 enrolled students and an open session; 50 load-test students exist
Expected result	The roster loads within 3 s; 95% of API requests complete within 2 s under 50 concurrent users.
Automated tests	`attendance.test_performance.ConcurrentLoadTests.test_tc11_50_concurrent_roster_requests_within_2_seconds`, … `test_nfr001_mixed_api_requests_under_concurrent_load`, and three `RosterPerformanceTests` (NFR-002)
Actual result	Single roster load 0.006 s. 50 concurrent: roster p95 0.229 s, mixed requests p95 0.241 s; 100% within 2 s, no errors.
Status	PASS

1. Log in as `perf.faculty@attendease.com`; time one `GET …/sessions/{id}/roster` (80 students); expect under 3 s.
2. With 50 concurrent threads, send 500 roster requests; record each response time.
3. Log in 50 load-test students; with 50 threads, send 500 requests across dashboard, history, my-sections and me.
4. For each phase, expect no errors and a 95th percentile of 2 s or less.

TC-12 · Service-layer coverage of at least 70%

Field	Detail
Requirements	NFR-006
Type / priority	Quality / High
Preconditions	Full test suite passes
Expected result	Automated test coverage of the service-layer business logic is at least 70%.
Automated check	CI step `coverage report --include=<7 service files> --fail-under=70`
Actual result	Statements 93.9% (542/577); branches 92.3% (155/168); 320 of 320 tests passed.
Status	PASS

1. Run `coverage run --branch --source=users,academic,attendance,corrections,analytics manage.py test`.
2. Run `coverage report` and `coverage json` on the 7 service files: `users/services.py`, `users/security.py`, `academic/services.py`, `attendance/services.py`, `corrections/services.py`, `corrections/notifications.py`, `analytics/services.py`.
3. Expect statement and branch coverage each at 70% or more.

TC-13 · Student cannot access another student's record

Field	Detail
Requirements	SEC-004
Type / priority	Security / High
Preconditions	Ananya (Student A) and Rohan (Student B) are both in CS341-A; Rohan's latest record id is known
Expected result	403 Forbidden (or an empty or filtered result) is returned; Student A cannot read or change a record outside her own data.
Automated tests	`corrections.tests.CorrectionSubmissionTests.test_another_students_record_is_not_disclosed`, `attendance.test_audit.AttendanceAuditTests.test_history_is_scoped_to_student_assigned_faculty_and_admin`, `attendance.test_dashboard.StudentDashboardTests.test_student_only_sees_enrolled_courses`, `attendance.test_sessions.RosterTests.test_fr015_roster_visible_to_assigned_faculty_and_admin_only`
Actual result	Correction on Rohan's record 404, nothing disclosed; Rohan's record history 404; section roster 403; Ananya's own history has only her rows; no request created in the database.
Status	PASS

1. Log in as `ananya@attendease.com`.
2. `POST /api/corrections` with Rohan's `record_id`; expect 403 or 404, with no data about the record.
3. `GET /api/attendance/sessions/{session}/records/{rohan}/history`; expect 403 or 404.
4. `GET /api/attendance/sessions/{session}/roster`; expect 403.
5. `GET /api/attendance/dashboard/history`; expect only her own rows.
6. In the database, expect no correction request on Rohan's record.

4. Traceability matrix

49 of the 52 SRS requirements are verified by a passing test case or automated test. NFR-005 is met structurally; NFR-008 and SEC-005 are deployment requirements that these tests do not cover. Test counts are automated tests in the suite (a test can cover several requirements).

Requirement	Summary	Test case	Automated tests	Status
FR-001	Register with name, email and role	TC-01	3	Pass
FR-002	Email uniqueness at registration	–	1	Pass
FR-003	Log in with email and password	TC-02	2	Pass
FR-004	Role-based access on every protected endpoint	TC-09	4	Pass
FR-005	Log out, invalidating the token	–	3	Pass
FR-034	Reset a forgotten password via email	–	3	Pass
FR-020	Attendance percentage per enrolled course	TC-06	5	Pass
FR-021	Session-wise attendance history	TC-06	5	Pass
FR-022	Flag courses below the threshold	–	1	Pass
FR-023	Filter history by date range	–	2	Pass
FR-006	Create and edit courses	–	8	Pass
FR-007	Create and edit sections	–	7	Pass
FR-008	Enroll students into a section	–	9	Pass
FR-009	Weekly timetable per section	–	8	Pass
FR-010	Bulk enrollment via CSV	–	7	Pass
FR-011	No duplicate section codes in a course	–	5	Pass
FR-012	Deactivate a course or section	–	8	Pass
FR-013	Assign a faculty member to a section	TC-03	8	Pass
FR-014	Start a session for a scheduled period	TC-04	10	Pass
FR-015	Roster for the active session	–	5	Pass
FR-016	Mark each student present or absent	TC-04	8	Pass
FR-017	Bulk-mark all present in one action	TC-04	5	Pass
FR-018	Close / finalize a session	–	5	Pass

Requirement	Summary	Test case	Automated tests	Status
FR-019	Timestamp session creation and closure	–	4	Pass
FR-037	No duplicate records per student and session	TC-05	15	Pass
FR-038	Edit within a window after closure	–	22	Pass
FR-024	Student raises a correction request	TC-07	20	Pass
FR-025	Email the faculty member on a new request	TC-07	16	Pass
FR-026	Pending-request queue for faculty	–	19	Pass
FR-027	Approve a request, updating the record	TC-08	18	Pass
FR-028	Reject a request with a reason	–	21	Pass
FR-029	Compute attendance percentage per course	TC-10	10	Pass
FR-030	Identify students below the threshold	–	15	Pass
FR-035	Per-course threshold set by an admin	–	7	Pass
FR-031	Defaulter report for a section	TC-10	9	Pass
FR-032	Section-wise summary report	–	8	Pass
FR-033	Export reports as CSV / PDF	–	6	Pass
FR-036	Filter reports by date range and section	–	3	Pass
NFR-001	95% of requests within 2 s at 50 users	TC-11	3	Pass
NFR-002	Roster for 80 students within 3 s	TC-11	4	Pass
NFR-003	No permanent deletion; auditable changes	–	20	Pass
NFR-004	No attendance for future dates	–	5	Pass
NFR-005	Each module independently testable	–	–	Met: each app has its own test modules
NFR-006	At least 70% service-layer coverage	TC-12	CI gate	Pass
NFR-007	Usable on desktop and tablet widths	–	1	Pass
NFR-008	Deployable with one docker-compose command	–	–	Not covered by these tests
SEC-001	Salted one-way password hashing	TC-01	1	Pass
SEC-002	Lock out repeated failed logins	TC-02	1	Pass
SEC-003	Server-side RBAC on every write endpoint	TC-09	8	Pass
SEC-004	Queries scoped to the user's own sections	TC-13	6	Pass
SEC-005	All traffic over HTTPS	–	–	Open: TLS to be set at the web server
SEC-006	Audit trail on approval and rejection	TC-08	59	Pass

5. Execution summary and sign-off

13 of 13 test cases passed on 10 November 2026, and no defects are open. The Test Plan exit criteria (§5.4) are met.

Measure	Result
Test cases executed / passed	13 / 13
System validation	43 API requests, 71 API assertions, 8 server-side checks; all passed
Automated suite	320 tests, 320 passed, 0 failed, 0 skipped
Service-layer coverage	93.9% statements, 92.3% branches (target 70%)
Requirement coverage	49 of 52 requirements verified; NFR-005 met structurally
Open defects	None
Open items	SEC-005 HTTPS to be enforced at the web server; TC-11 to repeat on PostgreSQL (D-09)

Role	Name	USN	Date
Project lead	Arundhati K	PES1UG24CS083	
QA lead	Bhoomika Dayanand Jituri	PES1UG25CS807	
Team member	Anshdeep Singh Sachdeva	PES1UG24CS070	
Team member	Atharva Ashish Vyas	PES1UG24CS093	